

Data-Driven CFD Models

Shanghai YK Pao School Cindy
Chen

Summary

Computational Fluid Dynamics (CFD) is an essential tool for studying fluid flow and has been widely applied in various fields, including engineering, aerospace, automotive design, and climate research. The data generated from CFD simulations typically contain complex spatiotemporal information, and the temporal prediction of this data holds significant practical importance in many applications. CFD solves the governing equations of fluid mechanics through numerical methods, providing detailed information about the flow field, including physical quantities such as velocity, pressure, and temperature. However, these data are often complex and exhibit high spatiotemporal dynamics, particularly in the prediction of turbulence, which makes temporal prediction a challenging task.

This study aims to employ Long Short-Term Memory (LSTM) networks to predict the velocity data of flow around a cylinder from CFD simulations, with the goal of improving the accuracy and reliability of predictions. By simplifying the model structure, Principal Component Analysis (PCA) and normalization techniques were used to effectively reduce the data dimensionality, enhancing both training efficiency and stability. During model training, appropriate parameter combinations were selected to enable the LSTM to effectively capture the dynamic features within the flow data. Experimental results, measured by the Root Mean Square Error (RMSE), show that the model performs well in predicting the velocity of flow around a cylinder. Compared with traditional prediction methods, the LSTM model demonstrates better performance in handling complex flow patterns, especially in capturing nonlinear characteristics and dynamic changes. Furthermore, when compared with the full LSTM model, the simplified version not only maintains comparable or slightly improved accuracy but also significantly reduces training time.

This report proposes the use of a simplified LSTM for temporal prediction of CFD velocity data in cylindrical flow, aiming to explore its effectiveness in capturing the dynamic features of the flow and its potential applications in fluid dynamics. Particularly in turbulence prediction, this approach offers a new perspective for fluid dynamics research by saving computational resources and improving prediction efficiency.

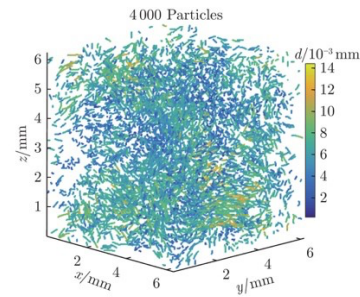
Keywords: Computational Fluid Dynamics, Long Short-Term Memory, Temporal Prediction, Data-Driven, Turbulence Prediction

Table of Contents

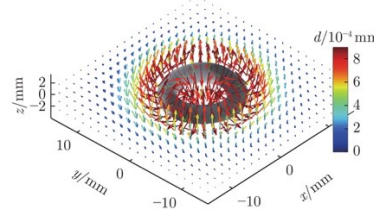
I. RESEARCH METHODS AND OBJECTIVES.....	5
1) Data Source.....	5
2) PCA Reduction.....	5
3) LSTM Model Construction	7
II. MAIN RESEARCH.....	7
1) Data Processing: Combining PCA And Normalization.....	7
2) LSTM Model Structure	8
3) Data Transformation for Supervised Learning.....	10
4) Model Training And Prediction	10
5) Evaluation Metrics	11
III. RESEARCH RESULTS AND ANALYSIS	12
IV. MAJOR INNOVATIONS	15
1) Simplified Model Architecture:	15
2) Application of Deep Learning in Turbulence Prediction:	15
3) Efficient Training and Optimization:	16
4) Data Preprocessing Strategy:	16
5) Dynamic Feature Capture:	16
V. CONCLUSION	16
VI. Existing Problems And future expectations	17
VII. References	18
VIII. APPENDIX	19

INTRODUCTION

Among various Computational Fluid Dynamics (CFD) problems, turbulence simulation—using numerical methods and models to simulate and predict the behavior of turbulent flows—presents challenges. As shown in Figure 1, the three-dimensional vortex motion and two-dimensional velocity distribution visualization vividly reflect the randomness and multi-scale characteristics of turbulence, with particles exhibiting irregular distribution and complex vortex structures. Turbulence is one of the most complex phenomena in fluid dynamics, characterized by its high instability, strong randomness, and intricate vortex structures, making temporal prediction of flow data particularly difficult.



(a) 各向同性强制湍流场中的三维运动轨迹



(b) 涡环流场三维速度矢量

Figure 1: Turbulence Simulation: Visualization of Three-Dimensional Vortex Motion and Two-Dimensional Velocity Distribution
(Turbulence Simulation - Google Search)

Specifically, for the flow around a cylinder, this problem has attracted considerable attention due to its complex flow characteristics and practical importance. The flow around a cylinder involves the generation of complex vortices as fluid flows past the cylinder, including the formation of boundary layers, vortex shedding, and the time-varying nature of the flow field. These nonlinear and time-varying characteristics significantly increase the difficulty of temporal prediction. Traditional methods such as linear regression and ARIMA models yield limited results when applied to such highly nonlinear flow data, thus necessitating the exploration of new approaches to improve predictive performance.

In recent years, deep learning techniques, particularly Long Short-Term Memory (LSTM) networks, have gained attention due to their ability to capture long-term dependencies in time-series data. By applying the LSTM model, the goal is to effectively handle the velocity data of CFD simulations of flow around a cylinder,

thereby enhancing prediction accuracy and reliability. This approach not only offers new insights into understanding complex turbulent phenomena but also opens up new directions for practical applications in fluid dynamics.

I. RESEARCH METHODS AND OBJECTIVES

This study applies Long Short-Term Memory (LSTM) networks to perform temporal prediction of velocity data for flow around a cylinder, generated through CFD simulations. The specific research process includes data collection, preprocessing, model construction, and visualization.

1) Data Source

The data used in this study comes from Prof. Xiao Dunhui's research group titled "Nonlinear Reduced-Order Model Research Based on Diffusion Mapping Fluid Learning." This data was obtained through Computational Fluid Dynamics (CFD) simulations. These simulations analyze the behavior of fluid flow, generating velocity information at different regions for various time points. The dataset consists of 102 time steps, with the first 92 time points $(t_0, t_1, \dots, t_{91})$ used for model training. The data for the subsequent 9 time points $(t_{92}, t_{93}, \dots, t_{100})$ is used for prediction, and the results are compared with the actual data to compute the error and analyze the model's performance.

2) PCA Reduction

Dimensionality reduction is a crucial step in data processing and analysis, especially when dealing with high-dimensional data. While high-dimensional data can provide rich information, it also increases computational complexity and the risk of overfitting. Therefore, dimensionality reduction serves as an important technique to mitigate these issues.

As shown in Figure 2, dimensionality reduction techniques can significantly reduce computational complexity, increase processing speed, and shorten both training and prediction times. Additionally, dimensionality reduction can help remove noise, allowing the model to focus on key information, thereby improving its predictive performance and generalization ability. In the subsequent visualization steps of this study, dimensionality reduction helps reveal the

underlying structure and patterns in the data, making it easier to identify and interpret relationships between variables.

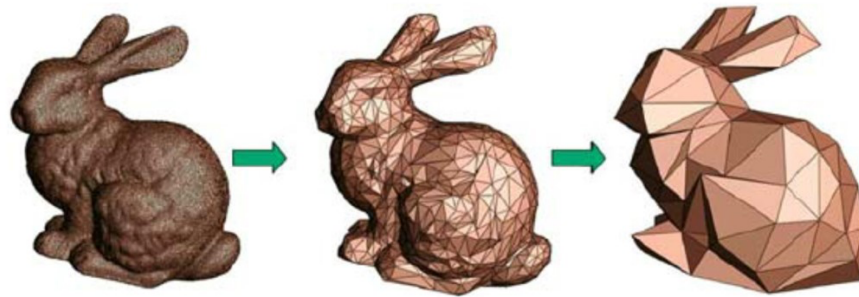


Figure 2: Diagram of Dimensionality Reduction Techniques
(Graphical illustration of model order reduction - Google Search)

Principal Component Analysis (PCA), a linear dimensionality reduction technique used in this study, works by computing the covariance matrix of the data and extracting the feature vectors that retain the maximum variance of the data. As shown in Figure 3, in the original three-dimensional data space, "Gene 1," "Gene 2," and "Gene 3" represent the axes, with each point corresponding to a sample. Through PCA, the original high-dimensional data is projected onto a two-dimensional plane (represented by PC1 and PC2 axes). PCA reduces the dimensionality of the data while retaining as much variance as possible. This linear dimensionality reduction technique is particularly useful in highdimensional biological data processing, helping to better understand the underlying structure and patterns in the data.

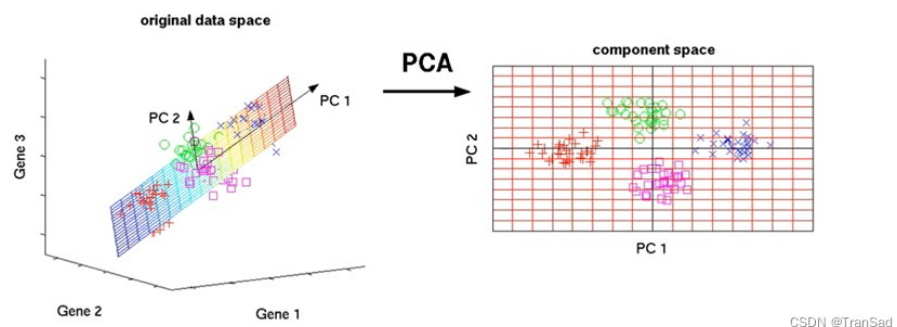


Figure 3: Illustration of PCA Dimensionality Reduction
(PCA Dimensionality Reduction - Google Search)

3) LSTM Model Construction

During the model construction phase, a Sequential API is used to build an LSTM network designed to process time-series data. The time step for the input layer is set to 10, meaning the model observes 10 consecutive time points at each step, with the feature dimension for each time step corresponding to the data processed through PCA.

The LSTM network architecture includes 200 neurons, which aim to enhance the model's memory capability and enable it to extract latent patterns from long sequences of data. To further improve the model's generalization ability, a Dropout layer is added after the fully connected layer to reduce the risk of overfitting. This LSTM architecture allows the model to capture the dynamic features of time-series data more effectively, thereby improving prediction performance.

II. MAIN RESEARCH

1) Data Processing: Combining PCA And Normalization

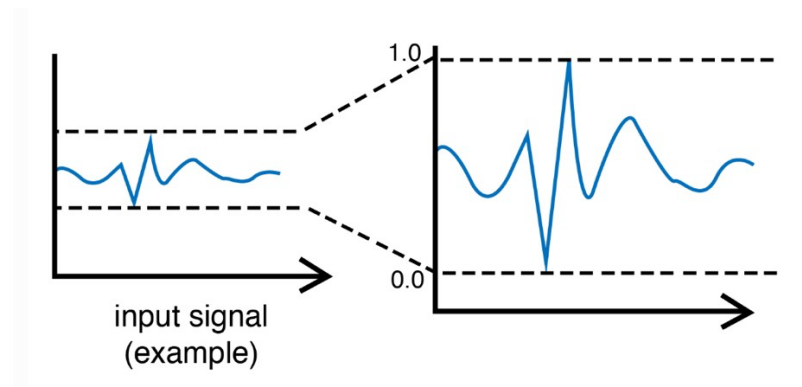


Figure 4: Principle of MinMaxScaler
(MinMaxScaler - Google Search)

Through data preprocessing, we can significantly improve the performance and stability of the model. Principal Component Analysis (PCA) reduces the dimensionality of the data to 10, which helps extract the most important information while reducing model complexity and computational costs. This not only improves training efficiency but also mitigates the risk of overfitting.

Additionally, MinMaxScaler is applied for normalization. As shown in Figure 4, the figure illustrates the principle of MinMaxScaler, using an example input signal before and after normalization. In the left panel, the input signal exhibits fluctuating patterns along the time axis. To handle such data effectively, MinMaxScaler transforms the signal so that it lies within a standard range of 0 to 1, as shown in the right panel. This normalization step is essential for stabilizing and improving the convergence of the subsequent model training process, making it one of the key preprocessing steps in machine learning. After the model has made predictions, the process of inverse normalization and re-projection ensures that the predicted values are converted back to the original data scale, enhancing the interpretability of the results.

2) LSTM Model Structure

The simplified LSTM model structure includes an input layer, one hidden LSTM layer, and an output layer. Specifically, the input layer accepts the data processed by PCA, while the LSTM hidden layer captures the dynamic changes in the timeseries data. The fully connected output layer generates the predicted results. In this study, a single LSTM hidden layer with 200 hidden units is employed.

The mathematical formulation for each component of the LSTM is as follows:

Forget Gate: The forget gate determines which information from the previous cell state $C^{(t-1)}$ should be forgotten:

$$f_t = \text{sigmoid}(w_f[x^{(t)}, h^{(t-1)}] + b_f)$$

Here, w_f is the weight matrix for the forget gate, which linearly transforms the input $x^{(t)}$ and the previous hidden state $h^{(t-1)}$, while b_f is the bias term. The sigmoid activation function generates a value between 0 and 1, where a value closer to 1 indicates that the information is retained, and a value closer to 0 means it is forgotten.

Output Gate: The output gate determines how much of the current cell state C^t should be passed to the next layer or output:

$$O_t = \text{sigmoid}(w_o[x^{(t)}, h^{(t-1)}] + b_o)$$

Like the forget gate, the output gate uses the sigmoid activation function to control the degree of output at each time step.

Input Gate: The input gate decides which information from the current input x^t and the previous hidden state h^{t-1} should be stored in the cell state C^t :

$$I_t = \text{sigmoid}(w_i[x^{(t)}, h^{(t-1)}] + b_i)$$

The sigmoid function provides the weight for selecting the input information.

Cell State Update: The current cell state C^t is updated using the forget gate and the input gate:

$$C^{(t)} = f_t C^{(t-1)} + I_t \tanh([x^{(t)}, h^{(t-1)}] + b_c)$$

Here, the previous cell state C^{t-1} is filtered by the forget gate, and new information, generated by the input gate and processed through a $\tanh$ function, is added.

Hidden State: The current hidden state h^t is calculated by the output gate and the current cell state:

$$h^{(t)} = O_t \tanh(C^{(t)})$$

The hidden state is then passed to the next time step or the output layer.

In summary, the simplified LSTM model is designed to capture long-term dependencies in the time-series data. The architecture enables the model to process and predict the flow characteristics effectively by learning the underlying temporal patterns.

Below is the diagram of the simplified LSTM structure (see Figure 5)

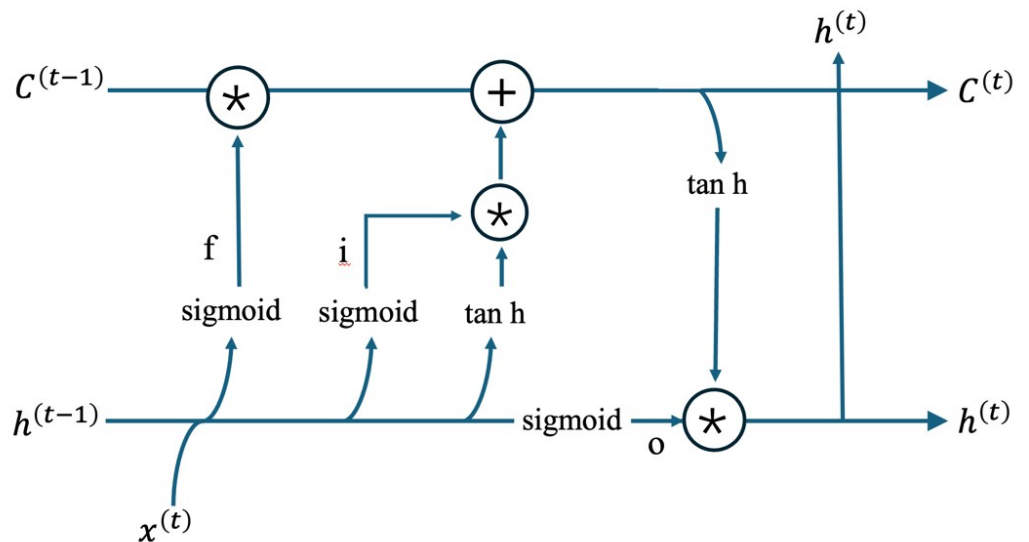


Figure 5: Simplified LSTM Structure Diagram

3) Data Transformation for Supervised Learning

For the data processing stage, the data is transformed into a supervised learning format. This involves re-organizing the time-series data into input-output pairs, enabling the model to learn how to predict future values from past observations.

Specifically, the input data X represents observations from the past time steps, and the output data y corresponds to future values.

To achieve this, a sliding window method is used, where a fixed-size window moves along the data to generate input-output pairs. This allows the model to learn how past observations predict the next time step's value. In this case, each sample consists of a fixed number of time steps, and the goal is to predict the value of the next time step.

4) Model Training And Prediction

The first 92 time points $(t_0, t_1, \dots, t_{91})$ are used as the training set, while the remaining 9 time point $(t_{92}, t_{93}, \dots, t_{100})$ form the test set. This setup ensures that the model has enough information to learn effectively during the training phase.

By enumerating various combinations of parameters, including the number of neurons in the LSTM, learning rate, dropout rate, and training epochs, the most suitable parameter combination is identified. The parameter combinations considered are as follows:

$lstm_units = [25, 50, 75, 100, 125, 150, 175, 200, 225, 250]$

$learning_rates = [0.00000001, 0.0000001, 0.000001, 0.00001, 0.0001, 0.001, 0.01, 0.1]$

$dropouts = [0.0000001, 0.000001, 0.00001, 0.0001, 0.001, 0.01, 0.1, 0.2, 0.5]$

$batch_sizes = [8, 16, 32, 64, 128, 256]$

$epoch = [1, 5, 20, 30, 50]$

After testing these combinations, the optimal parameter configuration was determined to be 200 neurons, a learning rate of 0.0001, dropout of 0.001, and 50

epochs. During each training epoch, the model adjusts its weights using an optimization algorithm to gradually improve prediction accuracy.

The Adam optimizer is used to accelerate the model's convergence, and the Mean Squared Error (MSE) is selected as the loss function. During model evaluation, Root Mean Squared Error (RMSE) is computed to assess the difference between the predicted and actual values. RMSE serves as a reliable metric for evaluating the model's predictive accuracy.

The overall approach, including layered model construction and appropriate regularization, enables the LSTM model to effectively handle complex timeseries data, leveraging its strength in capturing long-term dependencies while ensuring stability and high prediction performance.

5) Evaluation Metrics

After the model is trained, the test set is used for prediction, and RMSE is computed as an evaluation metric to quantify the prediction performance. The formula for RMSE is given by:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_{\text{real}} - y_{\text{predicted}})^2}$$

where y_{real} is the true value for the i data point, $y_{\text{predicted}}$ is the predicted value for the i data point, and n is the number of data points in the test set. A smaller RMSE indicates better predictive performance.

Additionally, animations comparing the actual and predicted velocity values are created to visually assess the model's prediction capabilities and demonstrate the temporal characteristics of the flow. Moreover, error animations are generated by subtracting the predicted values from the actual values to further analyze the model's performance at different time points.

III. RESEARCH RESULTS AND ANALYSIS

In this study, the final model's Root Mean Square Error (RMSE) was found to be 0.06177.

Typically, an RMSE value below 0.1 indicates a highly accurate prediction, a value between 0.1 and 0.2 suggests relatively high precision with some minor deviations, while an RMSE above 0.2 reflects weaker predictive performance. Therefore, with an RMSE of 0.06177, the model demonstrates excellent predictive capability, confirming its potential for accurate forecasting. This outcome provides a highly reliable foundation for subsequent applications.

To visually demonstrate the model's predictive capabilities, comparison charts of actual and predicted velocities were created. These charts clearly show the model's performance at different time steps, as well as the distribution of overall prediction errors. In addition, animations of velocity changes were generated to further highlight the temporal features of the flow, enhancing the visualization of the results.

The following figures show the velocity field animations derived from the original data in VTU format. The use of VTU files is due to their standardization in storing computational results, particularly for fluid dynamics simulations. VTU files are highly effective for representing grid and point data, including quantities like velocity and pressure, and they are particularly suited for complex geometries. This format allows for efficient extraction and visualization of flow field characteristics in later analysis.

Each image corresponds to the velocity distribution at a specific time step $(t_{92}, t_{93}, \dots, t_{100})$ for the cylinder wake flow. The images on the left show the actual data, while the ones on the right present the predicted velocity distribution obtained through the simplified LSTM model.

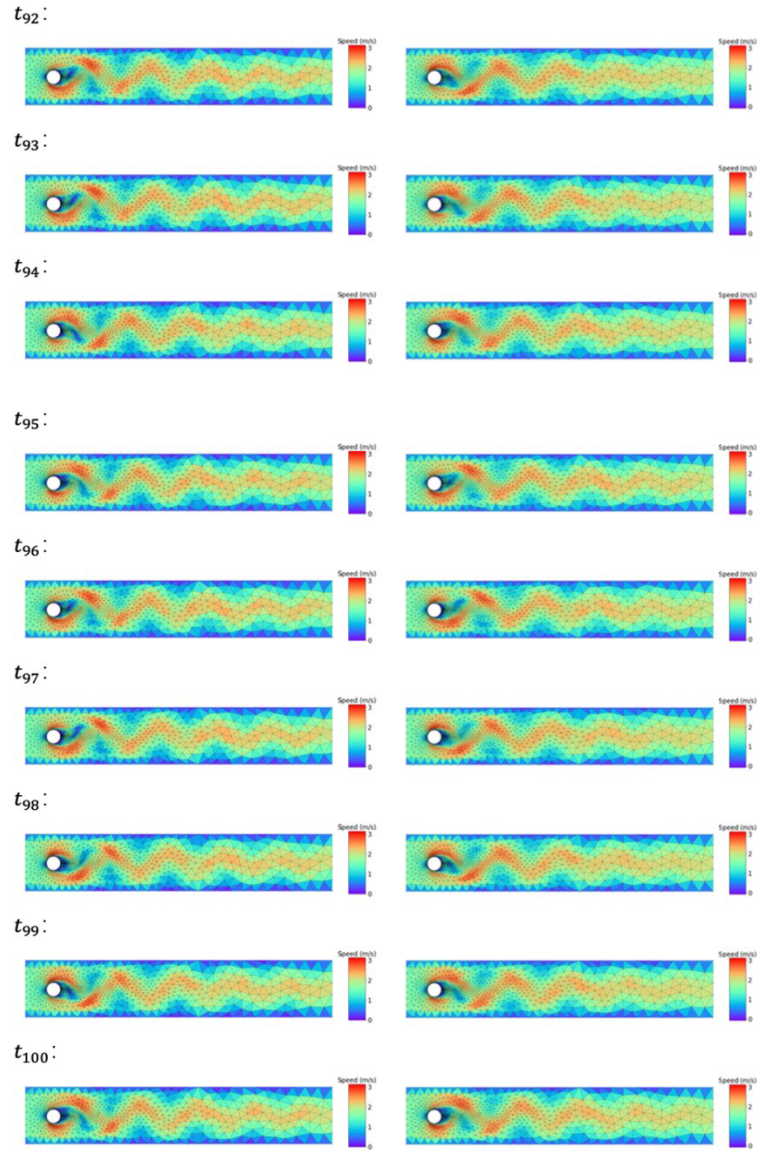


Figure 6: Velocity Distribution of Cylinder Wake Flow – Actual vs. Predicted Data

The following are the error distribution maps obtained by subtracting the predicted data from the actual data. These error maps represent the velocity distribution errors at the corresponding time steps $(t_{92}, t_{93}, \dots, t_{100})$, providing a direct and clear visual representation of the model's performance:

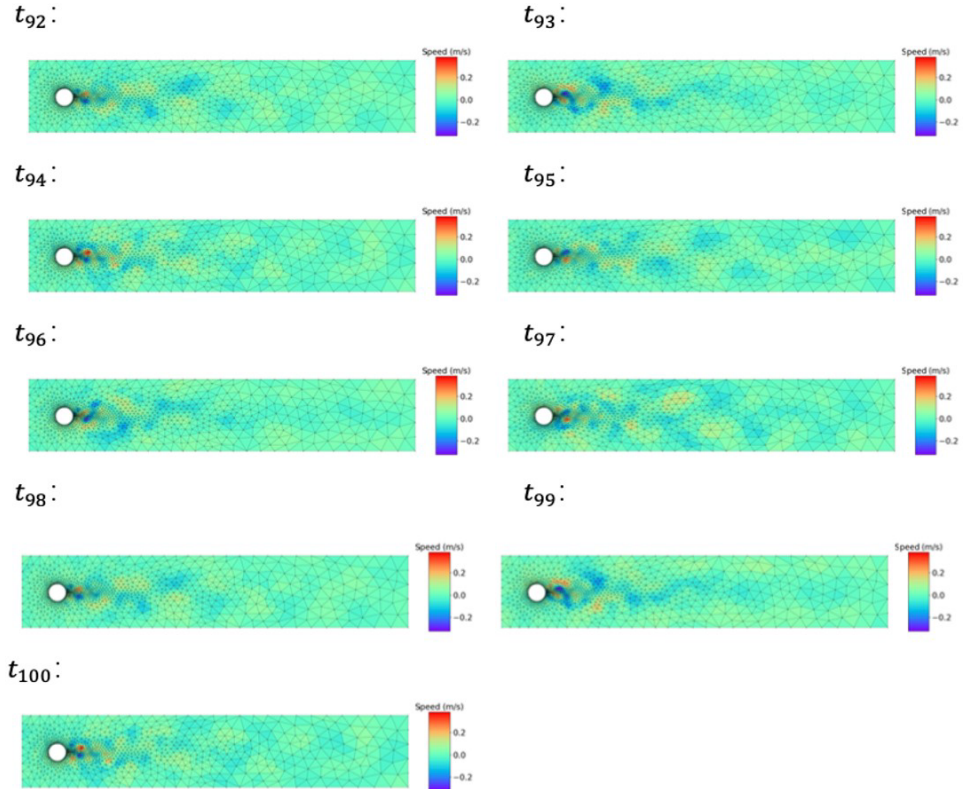


Figure 7: Error Maps – Difference between Actual Data and Predicted Data

By comparing the model's predictions with the actual observed data (as shown in Figure 6), it is evident that the two sets of data align closely in overall trend. The images illustrate the velocity distribution of a two-dimensional flow field.

The central region clearly displays a high-speed flow path formed as the fluid flows around the cylinder. This area exhibits faster flow rates, marked by warm colors (orange and red), representing the primary flow channels or high-speed regions.

The colors transition to cooler hues as we move to the sides, indicating lower speed areas. This characteristic speed distribution is highly consistent with the typical flow patterns observed in cylinder wake flow.

Additionally, some vortex or recirculation areas can be seen within the flow field. These regions display more complex flow dynamics and velocity distributions.

Furthermore, in the error maps shown in Figure 7, large areas of green indicate that the prediction errors are minimal, as reflected in the corresponding color scale. This strongly suggests that the model has been able to capture the complex velocity distribution features of the cylinder wake flow with a high degree of accuracy. The LSTM model demonstrates excellent performance in handling such time-series flow field data, showcasing both high predictive precision and efficiency.

However, there is also a noticeable region where prediction errors are more significant: the flow field behind the cylinder. This area presents more challenges in prediction, and the errors are concentrated in this region, which is more difficult for the model to predict accurately. This discrepancy may be attributed to the inherent complexity and turbulence in the wake flow region, which is more sensitive to small variations in the flow field and harder to model effectively.

IV. MAJOR INNOVATIONS

In this study, I proposed a simplified LSTM-based time-series prediction model for velocity data generated by CFD simulations of cylinder wake flow. Compared to traditional methods, the main innovations of this work are reflected in the following aspects:

1) Simplified Model Architecture:

The model utilizes a simplified LSTM structure without the inclusion of Attention and Concat layers. Although the Attention mechanism is commonly used to enhance the model's ability to focus on key information, repeated experiments in this study showed that incorporating Attention only marginally improved the model's RMSE to 0.09, with no significant improvement in prediction accuracy. In fact, omitting Attention led to a simpler model with reduced computational complexity and a shorter training time, which is of great significance for timeseries forecasting in real-time or high-frequency data. This design simplifies the overall architecture while maintaining a high level of predictive performance.

2) Application of Deep Learning in Turbulence Prediction:

The application of the LSTM model to turbulence prediction demonstrates the potential of deep learning techniques in computational fluid dynamics (CFD). The model is capable of significantly saving computational resources while capturing the non-linear characteristics of flow data more efficiently, which is a key challenge in turbulence modeling. This marks an important step toward leveraging deep learning in fluid dynamics research, offering more efficient solutions for complex flow predictions.

3) Efficient Training and Optimization:

The study involved systematic parameter adjustments and an optimized training process, which improved the model's prediction accuracy. By exploring different combinations of parameters such as the number of LSTM neurons, learning rates, dropout rates, and epochs, the model was fine-tuned to achieve the best performance. This iterative optimization strategy contributed to the enhancement of the model's overall reliability and efficiency.

4) Data Preprocessing Strategy:

A dual preprocessing approach combining Principal Component Analysis (PCA) and MinMaxScaler was employed to reduce overfitting and enhance the model's ability to capture complex flow patterns. PCA effectively reduces the dimensionality of the data, making it easier for the LSTM to focus on the most relevant features, while MinMaxScaler ensures that the data is normalized to the appropriate range. Together, these preprocessing techniques improve the model's stability, generalization, and predictive accuracy.

5) Dynamic Feature Capture:

Despite the simplification of the model's structure, the LSTM network was still able to effectively capture the dynamic features of the flow data. This demonstrates the potential of LSTM in fluid dynamics applications, particularly in predicting time-series data that involve complex, time-varying patterns. The ability of the LSTM model to retain long-term dependencies and adapt to nonlinear flow behaviors, even with a simplified architecture, highlights its applicability in CFD simulations and turbulent flow predictions.

V. CONCLUSION

This study successfully applied Long Short-Term Memory (LSTM) networks to perform time-series prediction on the velocity data of cylinder wake flow generated by Computational Fluid Dynamics (CFD) simulations, yielding positive results. Through systematic data processing and model training, the LSTM model demonstrated outstanding performance in both predictive accuracy and computational efficiency, particularly in handling turbulence data, which highlights the potential of deep learning in CFD applications.

By simplifying the LSTM model structure, the training time was significantly reduced to approximately 2 minutes per run, effectively minimizing computational resource consumption. Furthermore, the integration of Principal Component Analysis (PCA) and MinMaxScaler lowered model complexity, leading to a notable decrease in training time. The final model achieved a Root Mean Squared Error (RMSE) of 0.06177, demonstrating its high accuracy and reliability in flow prediction.

In conclusion, the LSTM-based model proved to be an efficient and reliable tool for time-series prediction in CFD simulations, especially in the context of turbulence. The model's simplified architecture, combined with advanced preprocessing techniques, provides a solid foundation for future work in predicting and analyzing complex flow dynamics with high accuracy and efficiency.

VI. Existing Problems And future expectations

Although the simplified LSTM model demonstrated good predictive performance with the current dataset, it may face certain limitations when dealing with more complex flow patterns or higher-dimensional data. To address these potential issues, future research could consider incorporating additional model optimization strategies to further improve prediction accuracy and stability. For example, more complex network architectures, such as multi-layer LSTM, CNN-LSTM hybrids, Graph Neural Networks (GNN), and Transformers, could be explored. Nonlinear dimensionality reduction techniques, such as t-Distributed Stochastic Neighbor Embedding (t-SNE) and autoencoders, could also be investigated. These approaches would help enhance the model's ability to capture the complex dynamics of fluid flow.

Moreover, the diversity and richness of the data are crucial factors influencing model performance. Collecting a broader range of sample data, particularly under varying flow conditions and different physical quantities, would improve the model's generalization ability and enhance its capacity to handle complex flow behaviors.

Additionally, based on the error maps generated in this study, it was observed that the errors were concentrated in the flow field behind the cylinder, which is more difficult to predict. Future research could focus on improving model performance in this region.

In conclusion, future research can build upon this work by exploring more efficient model designs and data processing techniques, further advancing the field of fluid dynamics.

VII. References

Kolarik, Thomas, and Gottfried Rudorfer. "Time Series Forecasting Using Neural Networks." *ACM SIGAPL APL Quote Quad*, vol. 25, no. 1, 1 Oct. 1994, pp. 86–94, <https://doi.org/10.1145/190468.190290>.

Lim, Bryan, and Stefan Zohren. "Time Series Forecasting with Deep Learning: A Survey." *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 379, no. 2194, 5 Apr. 2021, p. 20200209, arxiv.org/abs/2004.13408v1, <https://doi.org/10.1098/rsta.2020.0209>.

Wen, Xin, and Weibang Li. *Time Series Prediction Based on LSTM-Attention-LSTM Model*. Vol. 11, 1 Jan. 2023, pp. 48322–48331, <https://doi.org/10.1109/access.2023.3276628>. Accessed 4 Aug. 2023.

“深度学习的调参经验_lstm 学习率-CSDN 博客.” Csdn.net, 2020, blog.csdn.net/weixin_36670529/article/details/107205569. Accessed 30 Oct. 2024.

Gitee, Login . “Login - Gitee.” *Gitee*, 2024, gitee.com/nilol/pygeotutorial/blob/master/12-.ipynb. Accessed 30 Oct. 2024.

“实战演习(十)——通过 LSTM 训练天气污染程度预测模型_lstm 污染-CSDN 博客.” *Csdn.net*, 2019, blog.csdn.net/livan1234/article/details/88895820. Accessed 30 Oct. 2024.

S. Siامي-Namini, N. Tavakoli and A. Siامي Namin, "A Comparison of ARIMA and LSTM in Forecasting Time Series," 2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA), Orlando, FL, USA, 2018, pp. 1394-1401, doi: 10.1109/ICMLA.2018.00227. keywords: {Time series analysis;Forecasting;Predictive models;Autoregressive processes;Economics;Deep learning;Data models;Deep Learning, Long Short-Term Memory (LSTM), Autoregressive Integrated Moving Average (ARIMA), Forecasting, Time Series Data}

“将时间序列预测问题转换为 Python 中的监督学习问题_how to Convert a Time Series to a Supervised Learn-CSDN 博客.” *Csdn.net*, 2018, blog.csdn.net/u012735708/article/details/82772388. Accessed 30 Oct. 2024. 教 AI 的陶老师.

“3%E5%88%86%E9%92%9F%E5%B8%A6%E4%BD%A0%E6%90%9E%E6%B8 %85LSTM%20%E7%9A%84%E8%AE%A1%E7%AE%97%E8%BF%87%E7%A8

%8B%E5%92%8C%E8%AE%A1%E7%AE%97%E5%9B%BE.” *Bilibili.com*, 8 July 2023,
www.bilibili.com/video/BV1Sz4y1E7oL/?spm_id_from=333.337.searchcard.all.click&vd_source=10f1fcaca0ed1a8f83f4d5dcd9f8a8ac. Accessed 30 Oct. 2024.

graphical illustration of model order reduction - Google Search.
www.google.com/search?sca_esv=439ed1f28d78315f&sxsrf=ADLYWIKzg2vLDPNCDU5A2_JyflCelMvoZQ:1730334314993&q=graphical+illustration+of+model+order+reduction&udm=2&fbs=AEQNm0AaBOazvTRM_Uafu9eNJJzCjPEAP5HX2BE31zy5nlFpWmPFVdZpPWP9TZgZsLh8KhKr1t4BqqnkxEPPKxdHZbgNZFRlp4IsxHurGjzqus--mSIhKkd5lCniVzTwu_IR2pPq1qHVguyvE1zbnqCwx0KDW9Gnf0STc5LYzRB1ViYPWJvPYAuczqmxz36Nt6pW6mexskTH&sa=X&ved=2ahUKEwjlg8DRbeJAxUgdvUHHamsILMQtKgLegQIEhAB&biw=1710&bih=882&dpr=2.

Minmaxscaler - Google Search.
www.google.com/search?sca_esv=439ed1f28d78315f&sxsrf=ADLYWIKNtK3FMtn5uBomCt8fxVX4t81CMw:1730344548778&q=minmaxscaler&udm=2&fbs=AEQNm0AaBOazvTRM_Uafu9eNJJzCjPEAP5HX2BE31zy5nlFpWmPFVdZpPWP9TZgZsLh8KhKr1t4BqqnkxEPPKxdHZbgNznUQhjMOFGSiAV56JhjerLFI4C8SgNbLtEJHJpcH5O_XSpU7hDJzzgOizX5FG1nosRy7HHbCK988Xnodw7RcallhjrTRa9IFYOhspbm74CO4BSWSYlcrq66v_Mdetqdqaj5ow-Q&sa=X&sqi=2&ved=2ahUKEwj26zh07eJAxVEnK8BHXJoMokQtKgLegQIFBAB&biw=1710&bih=947&dpr=2#vhid=DfxYoIT2OdfpfM&vssid=mosaic.

VIII. APPENDIX

```
import pandas as pd
import numpy as np
import meshio
import torch
import tensorflow as tf
import os
```

```
from matplotlib import colors, tri as mtri from
matplotlib.animation import FuncAnimation from
mpl_toolkits.axes_grid1 import make_axes_locatable
import matplotlib.pyplot as plt from sklearn.manifold
import Isomap from torch_geometric.data import Data
```

```
from sklearn.preprocessing import LabelEncoder,MinMaxScaler from
sklearn.metrics import mean_squared_error from
tensorflow.keras.models import Sequential from tensorflow.keras.layers
import Dropout,Dense, LSTM, InputLayer from tensorflow.keras.losses
import MeanSquaredError from tensorflow.keras.metrics import
RootMeanSquaredError from tensorflow.keras.callbacks import
ModelCheckpoint from tensorflow.keras.optimizers import Adam
import tensorflow.compat.v1 as tf
```

```
import matplotlib.pyplot as plt from
math import sqrt
```

```
data_path = '/Users/cindychen/Desktop/英才计划重开! /fluidity/圆柱绕流/'
```

```
# create file path for all csv files. Store them in "files"
files = [] for i in range(102):  p1 = '12_'  p2 =
str(i)  p3 = '.csv'  suffix = p1 + p2 + p3  files +=
[data_path + suffix]
```

```
#Read csv from all the data path stored in files df_array = [] #
this is the array to store all df read from csv files
```

```
for i in range(len(files)): # 使用 files 的长度
df_array.append(pd.read_csv(files[i])['Speed']) # 只包含 Speed 列    df =
pd.DataFrame(df_array) ###

print("df shape is", df.shape)
```

```

#加 pca/scale+inverse from sklearn.preprocessing
import MinMaxScaler from
sklearn.decomposition import PCA

scaler = MinMaxScaler() scaled_data =
scaler.fit_transform(df.values)

pca = PCA(n_components=10) pca_result
= pca.fit_transform(scaled_data)

scaled_back = pca.inverse_transform(pca_result)

original_data = scaler.inverse_transform(scaled_back)

print(original_data.shape) print("Scale,
PCA and inverse Done")

#[[1],[2],[3],[4],[5]]- [6]
#[[2],[3],[4],[5],[6]]- [7] def
df_to_X_y (df, size=10):  df_as_np
= df.to_numpy()  X=[]  y=[]  for
i in range (len(df_array)-size):
row = df_as_np[i:i+size]
X.append(row)      label =
df_as_np[i+size]    y.append(label)
return np.array(X),np.array(y)

size = 10
X, y = df_to_X_y(pd.DataFrame(original_data), size)
print("X shape is", X.shape) print("y shape is",
y.shape) print("shift done")

#####

X_train, y_train = X[:82], y[:82]
X_test, y_test = X[81:90], y[81:90]

####

```

```

X_train = tf.convert_to_tensor(X_train)
y_train = tf.convert_to_tensor(y_train) X_test
= tf.convert_to_tensor(X_test) y_test =
tf.convert_to_tensor(y_test)

```

```

train_dataset = tf.data.Dataset.from_tensor_slices((X_train, y_train)).batch(1)

```

```

model = Sequential() model.add(LSTM(200,
input_shape=(X_train.shape[1], X_train.shape[2])))
model.add(Dense(1107)) model.add(Dropout(0.001))
model.compile(loss=MeanSquaredError(),
optimizer=Adam(learning_rate=0.0001),
metrics=[RootMeanSquaredError()]) epc=50 model.fit(train_dataset,
epochs=epc, batch_size=32,verbose=1)

```

```

y_predicted = model.predict(X_test)

```

```

rmse = sqrt(mean_squared_error(y_test, y_predicted)) print("RMSE:",
rmse)

```

```

def triangles_to_edges(faces):
    edges = torch.cat([faces[:, 0:2], faces[:, 1:3],
torch.stack([faces[:, 2], faces[:, 0]], axis=1)], dim=0) receivers =
tf.reduce_min(edges, axis=1) senders = tf.reduce_max(edges, axis=1)
packed_edges = tf.bitcast(tf.stack([senders, receivers], axis=1), tf.int64) df =
pd.DataFrame(packed_edges) unique_edges = df.drop_duplicates()
unique_edges = tf.convert_to_tensor(unique_edges) senders, receivers =
tf.unstack(unique_edges, axis=1) return (tf.concat([senders, receivers],
axis=0), tf.concat([receivers, senders], axis=0))

```

```

def get_dataset(path):

```

```

    vtu_files = sorted([f for f in os.listdir(path) if f.endswith('.vtu')]) print('文件数量为
', len(vtu_files))

```

```

    all_speed = []    for
filename in vtu_files:
    mesh = meshio.read(os.path.join(path, filename))    Speed
= torch.tensor(mesh.point_data["Speed"]).float()
all_speed.append(Speed)

    Speed_combined = torch.vstack(all_speed)    print('合并后的
矩阵规模为', Speed_combined.shape)    return Speed_combined,
mesh

def make_animation(mesh, speed_data):
    fig, ax = plt.subplots(1, 1, figsize=(20,
16))    ax.set_aspect('equal')
ax.set_axis_off()

    pos = mesh.points[:, :2]    faces =
mesh.get_cells_type('triangle')    triang =
mtri.Triangulation(pos[:, 0], pos[:, 1],
faces)

    norm = colors.Normalize(vmin=speed_data.min(), vmax=speed_data.max())

    mesh_plot = ax.tripcolor(triang, speed_data[0].numpy(), cmap='rainbow',
norm=norm, shading='flat')    ax.triplot(triang, 'ko-', ms=0.5, lw=0.3)

    divider = make_axes_locatable(ax)    cax =
divider.append_axes('right', size='5%', pad=0.05)    clb =
fig.colorbar(mesh_plot, ax=ax, cax=cax, orientation='vertical')
clb.ax.tick_params(labelsize=20)
clb.ax.set_title('Speed (m/s)', fontdict={'fontsize': 20})

    def update(frame):        ax.cla()        ax.set_aspect('equal')        ax.set_axis_off()
mesh_plot = ax.tripcolor(triang, speed_data[frame].numpy(), cmap='rainbow',
norm=norm, shading='flat')        ax.triplot(triang, 'ko-', ms=0.5, lw=0.3)        return
mesh_plot,    plt.close(fig)    ani = FuncAnimation(fig, update,
frames=range(speed_data.shape[0]), blit=True, repeat=True)

    return ani

```

```

data_path = "/Users/cindychen/Desktop/英才计划重开! /fluidity/vtu 文件/"

speed_data, mesh = get_dataset(data_path) ani
= make_animation(mesh, speed_data)

y_predicted_tensor = torch.tensor(y_predicted).float() # 变成 tensor ani_predicted
= make_animation(mesh, y_predicted_tensor)

error_data = y_test - y_predicted_tensor
error_data_numpy = error_data.numpy()
error_data_tensor = torch.tensor(error_data_numpy).float() # 先变成 numpy 再变成
tensor
    # EagerTensor 不能直接转换成 pytorch tensor, 因为它是 tensorflow 的数据结
    构, 和 pytorch tensor 不兼容
ani_error = make_animation(mesh, error_data_tensor)

from IPython.display import HTML from
IPython.display import display

display(HTML(ani.to_jshtml())) display(HTML(ani_predicted.to_jshtml()))
display(HTML(ani_error.to_jshtml()))

```

Acknowledgements

During the preparation of this research project, I have received essential guidance and support from many people, to whom I express my sincere gratitude.

First, I would like to thank my supervisor, Professor Xiaodunhui, for his selfless and invaluable guidance. He provided clear direction on the research background, recommended important literature, and offered constructive suggestions whenever I was uncertain about the scope of the study. All of his supervision was provided without any financial compensation, reflecting his genuine commitment to academic mentorship. His professionalism and encouragement have shaped both the quality of this research and my own growth as a student.

I am also grateful to my school advisor, Mr. Shen Yifan, for his practical and technical assistance. He gave detailed feedback during data collection, coding, and report writing. When I encountered difficulties, particularly with programming errors and data inconsistencies, he carefully explained solutions and encouraged me to persist. His patient guidance was crucial for overcoming the most challenging parts of the project.

In terms of division of work: the research topic was refined with Professor Xiao's advice; data collection and preprocessing were carried out by me, with guidance from Mr. Shen; data analysis and coding were mainly my responsibility, with technical corrections from Mr. Shen; interpretation of results was shaped through discussions with Professor Xiao; and drafting the report was my own task, later improved through comments from both advisors.

The process was not without challenges. For example, I initially struggled with missing data and programming errors, but with guidance I was able to restructure the dataset and correct the code. At one stage, my results did not align with theoretical expectations, and Professor Xiao suggested rechecking model assumptions, which helped me produce more reliable findings.

Finally, I would like to thank my family and classmates for their constant encouragement. Their emotional support gave me strength during difficult times and motivated me to complete the report with determination.

In conclusion, this project reflects a joint effort. Professor Xiao's academic guidance, Mr. Shen's technical assistance, and my own work in analysis and writing together made the research possible. I am truly grateful to everyone who contributed to this journey.